JavaScript Complete Beginner Reading Guide

From the first console message to a working class results page

Who this is for. A beginner who wants to understand code before memorizing syntax, and an instructor who wants examples to teach from. The first eight parts follow the JavaScriptTutorial.net path we used together. The later parts connect the language to browser pages and a complete project.

How to study. Read one lesson, predict its output, run it, change an input and explain the change in your own words. Run each stand-alone JavaScript example in a fresh browser console or script so repeated const and class names do not conflict. HTML examples go in an HTML file. Module and project examples need separate files as shown.

A practical limit. This guide covers the core language and common browser work, not every built-in API or obscure edge case. Later topics depend on the early ideas: values, conditions, loops, functions and objects.

Contents

Part 1 Getting started

Part 2 Values variables and types

Part 3 Operators and comparisons

Part 4 Decisions and loops

Part 5 Functions

Part 6 Objects and prototypes

Part 7 Classes

Part 8 Functions in more depth

Part 9 Array methods

Part 10 The DOM and the page

Part 11 Events and forms

Part 12 Timers promises and async await

Part 13 Fetching data

Part 14 Errors and modules

Part 15 Browser storage

Part 16 Complete class results project

Part 1 Getting started

Start with code you can run immediately. The console shows results and errors.

1. What JavaScript does

What it is. JavaScript stores values, calculates results, makes decisions and responds to events.

Real-life example. An online shop uses JavaScript to update the bill when you change the quantity.

How it works. A runtime executes your JavaScript. Browsers provide page and event APIs. Node.js runs JavaScript outside the browser and provides different APIs. JavaScript was created in 1995 and follows the ECMAScript standard. Browser features and the language itself are related but separate.

Code to try

```
const price = 100;
const quantity = 3;
console.log(price * quantity);
```

Expected result. 300

Why and when to use it. Use it when a page or application needs behavior or calculations.

Another way or a common risk. HTML can describe content and CSS can style it. Dynamic behavior needs executable logic.

2. Editor, browser and console

What it is. An editor helps you write code. A browser executes page scripts. Its console displays messages and errors.

Real-life example. The editor is your notebook. The runtime follows the instructions. The console is the result window.

How it works. For saved work, install VS Code from its official website. Create a folder containing index.html and app.js. No extensions are required for these examples. The console may also display undefined as the return value of console.log. That does not mean the printed message failed.

Code to try

```
console.log("Hello, world!");
console.log(5 + 3);
```

Expected result. Hello, world! | 8

Why and when to use it. Open browser developer tools with F12 or Inspect, then choose Console.

Another way or a common risk. Without an editor you can still practice in the console. An editor helps save and organize larger programs.

3. Hello world in a saved page

What it is. A script element tells the browser which JavaScript file to load.

Real-life example. Opening a class notice page can also run a small welcome script.

How it works. Create the two files in the same folder. The HTML comment belongs in HTML; the JavaScript comment and console.log belong in app.js. Open index.html in a browser and inspect the console. defer lets a classic external script execute after HTML parsing.

Code to try

```
<!-- index.html -->
<!doctype html>
<h1>Student portal</h1>
<script src="app.js" defer></script>

// Put this line in app.js:
console.log("Portal ready");
```

Expected result. The page shows Student portal. | The console shows Portal ready.

Why and when to use it. Use separate files when you want to save and grow a project.

Another way or a common risk. Console experiments work temporarily. A saved file makes the example reusable.

Practice and explanation

Your task. What does each line print? Which lines contain arithmetic?

Predict before running

```
console.log(10 + 2);
console.log("10 + 2");
console.log("10" + "2");
```

Expected output. 12 | 10 + 2 | 102

Why. Quotes create text. The middle line contains one string. The last line joins two strings.

Further reading https://www.javascripttutorial.net/

Part 2 Values variables and types

Values represent information. Variables and collections let you organize it.

4. Syntax: reading the punctuation

What it is. Syntax is the grammar JavaScript expects. A statement performs an action. An expression produces a value.

Real-life example. A recipe needs a readable order. Code needs names and punctuation that the runtime understands.

How it works. 40 + 10 is an expression whose value is 50. const mark = 40 + 10 is a declaration statement. Parentheses hold call arguments. Curly braces group blocks or define objects depending on context. Names are case-sensitive: mark and Mark differ. Reserved words such as return cannot be ordinary variable names. Semicolons explicitly end statements; JavaScript also has automatic semicolon insertion rules.

Code to try

```
// This comment explains the code
const mark = 40 + 10;
console.log(mark);
```

Expected result. 50

Why and when to use it. Use clear names, indentation and comments explaining the reason for a step.

Another way or a common risk. Incorrect syntax prevents a script from running. Poor formatting makes correct code harder to follow.

5. Variables: let and const

What it is. A variable is a name connected to a value. let allows reassignment. const requires an initial value and prevents reassignment.

Real-life example. A current score may change. A student ID normally stays the same during this task.

How it works. The equals sign assigns the right-hand result to the left-hand variable. It is not an equality question. A const object may still have mutable properties: const stops rebinding the variable. The next chapter explains equality operators.

Code to try

```
let score = 40;
score = score + 10;
const studentId = "S101";
console.log(score, studentId);
```

Expected result. 50 S101

Why and when to use it. Use const by default. Use let when you intend to assign a new value later.

Another way or a common risk. Hard-coded values repeated everywhere make updates and calculations difficult.

6. Scope: where a name is available

What it is. let and const belong to the block where you declare them. A block uses curly braces.

Real-life example. A note inside one classroom is available inside that classroom.

How it works. var is older syntax with function scope rather than block scope. Prefer let and const for new learning. Reading a let or const before its declaration runs causes a ReferenceError. Function declarations have different early-access behavior, discussed in Section 5.

Code to try

```
if (true) {
  let localScore = 50;
  console.log(localScore);
}
// localScore is unavailable here.
```

Expected result. 50

Why and when to use it. Use local variables so unrelated parts of a program do not interfere.

Another way or a common risk. A shared global variable can change unexpectedly when many functions write to it.

7. Data types: different kinds of values

What it is. Numbers calculate. Strings hold text. Booleans answer yes or no. Objects group information.

Real-life example. A record has a numeric mark, a text name and a true-or-false paid status.

How it works. JavaScript has seven primitive types: string, number, bigint, boolean, undefined, symbol and null. Object is the other type. Arrays and functions are objects with special behavior. Variables can be

assigned values of different types. typeof null returns "object" for historical reasons, although null is a primitive.

Code to try

```
const mark = 75;
const name = "Asha";
const paid = true;
console.log(typeof mark);
console.log(typeof name);
console.log(typeof paid);
```

Expected result. number | string | boolean

Why and when to use it. Choose values that match the meaning of your data.

Another way or a common risk. Treating "75" as a number without conversion can join text instead of adding.

8. undefined, null, BigInt and Symbol

What it is. undefined often means a value is missing. null is an explicit empty value. BigInt holds large integers. Symbols provide unique property keys.

Real-life example. A missing answer differs from an answer deliberately marked empty.

How it works. BigInt literals end in n. Arithmetic cannot directly mix BigInt and Number values. Two calls to Symbol create different symbols even if their descriptions match. You do not need BigInt or Symbol for ordinary marks in this deck.

Code to try

```
let answer;
const selectedStudent = null;
console.log(answer, selectedStudent);
console.log(10n + 2n);
console.log(Symbol("id") === Symbol("id"));
```

Expected result. undefined null | 12n | false

Why and when to use it. Use null when your design needs an explicit empty state. Learn BigInt and Symbol when their use arises.

Another way or a common risk. Ordinary numbers lose integer precision at very large sizes. Reused string keys may collide.

9. Numbers and conversion

What it is. Number represents whole and decimal numbers. Convert numeric text before doing numeric work.

Real-life example. A form can provide "50" as text even when it looks like a number.

How it works. NaN means not a valid numeric result and has typeof number. Infinity can result from 1 / 0. Binary floating-point cannot represent every decimal exactly: 0.1 + 0.2 is approximately 0.30000000000000004. Money often needs a deliberate integer-unit or decimal approach.

Code to try

```
const enteredMark = "50";
console.log(enteredMark + 10);
console.log(Number(enteredMark) + 10);
console.log(Number("hello"));
```

Expected result. 5010 | 60 | NaN

Why and when to use it. Convert and validate input before calculations. Number.isNaN(value) checks for NaN.

Another way or a common risk. The + operator may join text. Invalid conversions produce NaN and spoil later calculations.

10. Numeric separators and number bases

What it is. Underscores make long number literals readable. Prefixes describe binary or octal notation.

Real-life example. Write a large budget clearly. Read binary flags when a technical task uses them.

How it works. The underscore is part of the source-code notation, not part of the number. Number("100_000") is NaN. Binary uses digits 0 and 1. Octal uses digits 0 through 7. Both literals shown represent the decimal number ten.

Code to try

```
const budget = 100_000;
console.log(budget);
console.log(0b1010);
console.log(0o12);
```

Expected result. 100000 | 10 | 10

Why and when to use it. Use separators for readability and base prefixes for data naturally written in those bases.

Another way or a common risk. You can write the same values in decimal. These forms are optional ways to express numbers.

11. Booleans and truthiness

What it is. A boolean is true or false. Conditions also convert other values to a boolean meaning.

Real-life example. An empty input usually means no text was supplied.

How it works. Common falsy values include false, 0, -0, 0n, an empty string, null, undefined and NaN. Nonempty strings, empty arrays and empty objects are truthy. The string "false" contains text, so it is truthy.

Code to try

```
console.log(Boolean(""));
console.log(Boolean("hello"));
console.log(Boolean(0));
console.log(Boolean("false"));
```

Expected result. false | true | false | true

Why and when to use it. Use explicit comparisons when the exact rule matters.

Another way or a common risk. Assuming that the text "false" is false can allow a condition unexpectedly.

12. Strings: text and interpolation

What it is. A string is text in quotes. A template literal uses backticks and inserts values with ${...}.

Real-life example. A result message combines a student name with a calculated mark.

How it works. Single and double quotes both create strings. Backticks support interpolation and multiline text. String indexes start at zero. Strings are immutable: operations produce new strings rather than changing a character in place. length counts UTF-16 code units, which can differ from visible character counts for some symbols.

Code to try

```
const name = "Asha";
const mark = 75;
console.log(`${name} scored ${mark}`);
console.log(name.length);
console.log(name[0]);
```

Expected result. Asha scored 75 | 4 | A

Why and when to use it. Use strings for labels, names and messages.

Another way or a common risk. Without templates, use + to join text. Both work, but long messages become harder to read.

13. Objects: related details together

What it is. An object groups named properties. Dot notation uses a known name. Brackets can use a variable.

Real-life example. One student record contains name, mark and section.

How it works. student.field would search for a property literally called field. student[field] evaluates the variable and searches for mark. Assigning a new property adds it. delete student.mark removes the property. Reading a missing ordinary property usually returns undefined.

Code to try

```
const student = {name: "Asha", mark: 75};
console.log(student.name);
const field = "mark";
console.log(student[field]);
student.mark = 80;
console.log(student.mark);
```

Expected result. Asha | 75 | 80

Why and when to use it. Use an object when information belongs to one thing.

Another way or a common risk. Separate unrelated variables are harder to pass around and keep together.

14. Arrays: a list with numbered positions

What it is. An array stores an ordered list. Its first position is index 0.

Real-life example. A teacher has a list of marks for several students.

How it works. push adds an item at the end. pop removes and returns the last item. length is the array length. Indexes in this example run from zero to length minus one. const allows mutation of the array but prevents assigning a new array to the variable.

Code to try

```
const marks = [70, 45, 82];
console.log(marks[0]);
marks.push(90);
console.log(marks.length);
console.log(marks[3]);
```

Expected result. 70 | 4 | 90

Why and when to use it. Use arrays when you want to loop through a collection.

Another way or a common risk. Separate mark1, mark2 and mark3 variables make repetition and growth awkward.

15. Copying values and sharing objects

What it is. Copying a primitive gives a separate value. Copying an object variable gives access to the same object.

Real-life example. Two people can hold copies of the address of one shared student record.

How it works. Objects are not automatically cloned by assignment. For a simple flat record, {...first} creates a shallow copy. Nested objects inside that copy remain shared. JavaScript arguments are also passed by value, including reference values, as Section 5 explains.

Code to try

```
let a = 40;
let b = a;
b = 50;
console.log(a);
const first = {mark: 40};
const second = first;
second.mark = 50;
console.log(first.mark);
```

Expected result. 40 | 50

Why and when to use it. Remember sharing when you update objects or arrays.

Another way or a common risk. An accidental shared object can change in two places when you intended independent data.

16. var and modern variable scope

What it is. var is an older way to declare a variable. Its scope is a function, while let and const follow a block.

Real-life example. A loop counter declared with var can remain visible after the loop, even when you expected it to stay inside.

How it works. var declarations are initialized to undefined before the line executes. let and const bindings also exist in their scope but cannot be accessed before initialization. const stops reassignment, not mutation of an object it refers to.

Code to try

```
for (var oldIndex = 0; oldIndex < 2; oldIndex++) {}
console.log(oldIndex);
for (let newIndex = 0; newIndex < 2; newIndex++) {}
// newIndex is unavailable here
```

Expected result. 2

Why and when to use it. Read var when maintaining older code. Prefer const unless you plan to reassign, then use let.

Another way or a common risk. If you ignore scope, two unrelated parts of a script may read or change the same name. Use let and const for new code.

Try it yourself. What would happen if you removed the comment and logged newIndex?

Check your answer. A ReferenceError occurs because newIndex belongs to its for block.

17. String interpolation and template literals

What it is. A template literal is text in backticks. The ${...} form inserts the value of an expression.

Real-life example. A result message includes a name and a calculated mark.

How it works. The expression inside ${...} is evaluated first. The values are converted to text and placed in the resulting string. Quotes inside a template literal do not start new strings.

Code to try

```
const name = "Asha";
const mark = 70;
console.log(`${name} scored ${mark + 5}`);
```

Expected result. Asha scored 75

Why and when to use it. Use it when a message combines fixed text and changing values.

Another way or a common risk. Without interpolation, you can join text with +. Long messages then become harder to read and spacing mistakes become common.

Try it yourself. What prints if mark becomes 40?

Check your answer. Asha scored 45.

Practice and explanation

Your task. Predict all three outputs. Why can a const object change?

Predict before running

```
const student = {mark: 50};
const other = student;
other.mark = 60;
console.log(student.mark);
console.log("mark");
console.log([10, 20][0]);
```

Expected output. 60 | mark | 10

Why. Both variables reach the same object. const protects the variable binding. Quotes mean literal text and array indexing starts at 0.

Further reading https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Grammar_and_types

Part 3 Operators and comparisons

Operators calculate results and produce the true or false values that conditions use.

18. Arithmetic operators

What it is. + adds numbers. - subtracts. * multiplies. / divides. ** raises a number to a power.

Real-life example. A shop calculates price times quantity, then subtracts a discount.

How it works. Multiplication and division have higher precedence than addition and subtraction. Parentheses override the default grouping. Number division can return a fraction. The + operator also joins strings, so check input types.

Code to try

```
const bill = 100 * 3 - 20;
console.log(bill);
console.log(10 / 4);
console.log(2 ** 3);
console.log((10 + 2) * 3);
```

Expected result. 280 | 2.5 | 8 | 36

Why and when to use it. Use arithmetic for totals, averages and counts. Parentheses make grouping clear.

Another way or a common risk. You would need another calculation method or repeated manual work.

19. Remainder: %

What it is. The remainder operator gives what remains after division.

Real-life example. Put 7 students into pairs. One student remains.

How it works. For nonnegative whole numbers, n % 2 === 0 tests whether n is even. JavaScript % is a remainder operator, not a mathematical modulo operation for every negative input. Keep the simple positive examples in mind first.

Code to try

```
console.log(7 % 2);
console.log(8 % 2);
console.log(8 % 2 === 0);
```

Expected result. 1 | 0 | true

Why and when to use it. Use it to check even numbers or cycle through repeating positions.

Another way or a common risk. You can calculate a remainder manually, but % expresses the operation directly.

20. Assignment operators

What it is. = stores a value. += updates by adding. Similar forms include -=, *= and /=.

Real-life example. A teacher adds five bonus marks to the current score.

How it works. For this numeric example, mark += 5 is equivalent to mark = mark + 5. Assignment is an action, not a comparison. += can concatenate when strings are involved.

Code to try

```
let mark = 40;
mark += 5;
console.log(mark);
mark *= 2;
console.log(mark);
```

Expected result. 45 | 90

Why and when to use it. Use them when the update depends on the current value.

Another way or a common risk. mark = mark + 5 works too. Compound assignment is a shorter form.

21. Unary operators and ++

What it is. Unary operators work on one value. ++ adds one and -- subtracts one. Unary + converts to Number.

Real-life example. A ticket counter increases whenever a new person joins.

How it works. Postfix tickets++ produces the old value, then leaves tickets incremented. Prefix ++tickets increments and produces the new value. Unary minus negates a numeric value. Number(text) is often clearer than unary + for beginners.

Code to try

```
let tickets = 2;
console.log(tickets++);
console.log(tickets);
console.log(++tickets);
console.log(+"50");
```

Expected result. 2 | 3 | 4 | 50

Why and when to use it. Use simple increments in counters and loops.

Another way or a common risk. tickets = tickets + 1 works too. Avoid clever mixed expressions while learning.

22. Comparison: =, == and ===

What it is. = assigns. === compares value and type. == may convert types before comparing.

Real-life example. A mark of exactly 50 meets an at-least-50 pass rule.

How it works. Other comparisons include <, <= and >. !== means strict inequality. In this example === is false because one value is a number and the other is a string. Two separately created objects are not === even if their properties look equal.

Code to try

```
const mark = 50;
console.log(mark >= 50);
console.log(mark > 50);
console.log(mark === "50");
console.log(mark == "50");
```

Expected result. true | false | false | true

Why and when to use it. Prefer === and !== when checking equality. Use >= when equality should count.

Another way or a common risk. A loose comparison or wrong boundary can let the wrong value pass a check.

23. Logical operators: &&, || and !

What it is. && requires both conditions. || accepts either. ! reverses the boolean meaning.

Real-life example. A student may enter when registered and paid. A staff member may enter through another rule.

How it works. && and || short-circuit: JavaScript skips the second expression if the first already determines the result. They return operand values, not necessarily booleans. The examples use booleans for clarity. ! always returns a boolean.

Code to try

```
const registered = true;
const paid = false;
const staff = true;
console.log(registered && paid);
console.log(paid || staff);
console.log(!paid);
```

Expected result. false | true | true

Why and when to use it. Combine several rules into one condition.

Another way or a common risk. Nested if statements can express the same logic but may become repetitive.

24. Nullish fallback and optional access

What it is. ?? chooses a fallback only when the left value is null or undefined. ?. stops a property access at a missing object.

Real-life example. Zero bonus marks are valid and should not be replaced by a default bonus.

How it works. The left side of ?? is evaluated. If it is null or undefined, JavaScript evaluates and returns the right side. Optional chaining stops on null or undefined and produces undefined for that chain.

Code to try

```
const bonus = 0;
console.log(bonus ?? 5);
const student = {};
console.log(student.address?.city ?? "Unknown");
```

Expected result. 0 | Unknown

Why and when to use it. Use for optional fields and values where zero or empty text can be meaningful.

Another way or a common risk. Using || as a default would replace valid values such as 0 or an empty string. Direct access to a missing nested object can throw.

Try it yourself. What is 0 || 5, and why is it different?

Check your answer. It is 5 because 0 is falsy. 0 ?? 5 is 0.

Practice and explanation

Your task. Predict the four outputs and explain the equality boundary.

Predict before running

```
const mark = 50;
console.log(mark >= 50);
console.log(mark === "50");
console.log(7 % 2);
console.log(true && false);
```

Expected output. true | false | 1 | false

Why. Exactly 50 meets >= 50. Number and string differ under ===. Seven leaves one after pairing. Both sides of && must be true here.

Further reading https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Expressions_and_operators

Part 4 Decisions and loops

Conditions choose a path. Loops repeat a block while a rule allows them to continue.

25. if: perform an action conditionally

What it is. An if block runs when its condition has a truthy value.

Real-life example. Show a congratulations message only when the student passes.

How it works. With mark = 40, only Check complete prints. Code after the if continues normally. Parentheses surround the condition. Braces surround the instructions controlled by that condition.

Code to try

```
const mark = 75;
if (mark >= 50) {
  console.log("Congratulations");
}
console.log("Check complete");
```

Expected result. Congratulations | Check complete

Why and when to use it. Use if for an optional action.

Another way or a common risk. Without a condition, the action runs even when the rule is not met.

26. if…else: choose between two paths

What it is. if handles the true case. else handles the remaining case.

Real-life example. A student receives either Pass or Fail.

How it works. The two blocks are alternatives. A true condition executes the first block and skips the else block. At exactly 50, the result is Pass because >= includes equality.

Code to try

```
const mark = 40;
if (mark >= 50) {
  console.log("Pass");
} else {
  console.log("Fail");
}
```

Expected result. Fail

Why and when to use it. Use it when exactly one of two actions should happen.

Another way or a common risk. Two separate if statements need care to ensure the conditions cover both cases correctly.

27. else if: several possible outcomes

What it is. JavaScript checks branches in order and runs the first matching branch.

Real-life example. Marks can produce Excellent, Pass or Fail.

How it works. Order matters. If mark >= 50 came first, 85 would take that branch and never reach Excellent. Put the narrower higher threshold first in this grading example.

Code to try

```
const mark = 85;
if (mark >= 80) {
  console.log("Excellent");
} else if (mark >= 50) {
  console.log("Pass");
} else {
  console.log("Fail");
}
```

Expected result. Excellent

Why and when to use it. Use it when one value can fall into several categories.

Another way or a common risk. Several independent if statements could print both Excellent and Pass for 85.

28. Ternary: choose one value

What it is. condition ? valueIfTrue : valueIfFalse produces a value.

Real-life example. A short label depends on whether a mark passes.

How it works. Read the question mark as choose the first value if true, and the colon as otherwise. Nested ternaries can become difficult to read. A ternary is an expression, which means it produces a value.

Code to try

```
const mark = 75;
const result = mark >= 50 ? "Pass" : "Fail";
console.log(result);
```

Expected result. Pass

Why and when to use it. Use a ternary for a short two-way value choice.

Another way or a common risk. An if…else with assignments works equally well. Prefer it for longer actions.

29. switch: select a matching value

What it is. switch compares one value with several cases. break prevents continued execution into the next case.

Real-life example. A portal shows a different message for each role.

How it works. Case matching uses strict equality. default handles values without a matching case. switch is usually less natural for numeric ranges such as mark >= 80.

Code to try

```
const role = "teacher";
switch (role) {
  case "student":
    console.log("View marks");
    break;
  case "teacher":
    console.log("Enter marks");
    break;
  default:
    console.log("Sign in");
}
```

Expected result. Enter marks

Why and when to use it. Use switch for several exact alternatives of one expression.

Another way or a common risk. if…else works too. Missing break can accidentally run later case bodies.

30. while: test before repeating

What it is. A while loop checks its condition before every round. It may run zero times.

Real-life example. Send up to three reminder attempts while attempts remain.

How it works. After printing 3, attempt becomes 4. The condition 4 <= 3 is false, so the loop stops. This example prints reminders; it does not actually send messages.

Code to try

```
let attempt = 1;
while (attempt <= 3) {
  console.log("Reminder " + attempt);
  attempt++;
}
```

Expected result. Reminder 1 | Reminder 2 | Reminder 3

Why and when to use it. Use while when continuation depends on a condition.

Another way or a common risk. Repeated copied code cannot easily adapt to changing counts. Forgetting progress may cause an infinite loop.

31. do…while: run once before testing

What it is. do…while executes its body before checking the condition. It always runs at least once.

Real-life example. Show a menu once, then decide whether it needs to appear again.

How it works. The condition is false after the first body, yet the body has already run. A do…while statement ends with a semicolon. Avoid using it when zero executions should be possible.

Code to try

```
let attempts = 0;
do {
  console.log("Show menu");
  attempts++;
} while (attempts < 0);
```

Expected result. Show menu

Why and when to use it. Use it when one initial attempt is required before you can evaluate repetition.

Another way or a common risk. A while loop might skip the first action. You could perform that action separately, then loop.

32. for: start, check, update

What it is. A for loop places the starting value, continuation test and update together.

Real-life example. Print each student number from 1 through 3.

How it works. The initialization runs once. Then the condition runs, then the body, then the update. This cycle continues until the condition is false. The update does not happen before the first body.

Code to try

```
for (let number = 1; number <= 3; number++) {
  console.log("Student " + number);
}
```

Expected result. Student 1 | Student 2 | Student 3

Why and when to use it. Use for when a counter or array index controls repetition.

Another way or a common risk. A while loop can do the same work, but its start and update are farther apart.

33. Loop trace: every round counts

What it is. A loop keeps checking until its condition fails, even after it prints one matching value.

Real-life example. Find every even student number from 1 to 4.

How it works. Round 1: remainder 1, no print. Round 2: remainder 0, print 2. Round 3: remainder 1, no print. Round 4: remainder 0, print 4. Then i is 5 and the outer condition fails.

Code to try

```
for (let i = 1; i <= 4; i++) {
  if (i % 2 === 0) {
    console.log(i);
  }
}
```

Expected result. 2 | 4

Why and when to use it. Trace each value of i when a loop output is confusing.

Another way or a common risk. Stopping your mental trace after the first match misses later output.

34. break: stop the loop early

What it is. break exits the nearest loop immediately.

Real-life example. Stop searching when you find the first failing mark.

How it works. break does not return from a function. It exits the nearest loop or switch. Instructions after the loop can still run. This example deliberately asks for the first failing mark only.

Code to try

```
const marks = [80, 45, 30];
for (let i = 0; i < marks.length; i++) {
  if (marks[i] < 50) {
    console.log(marks[i]);
    break;
  }
}
```

Expected result. 45

Why and when to use it. Use break when your goal is already achieved.

Another way or a common risk. The loop would keep searching and could produce additional results.

35. continue: skip this round

What it is. continue skips the rest of the current loop body and proceeds to the next iteration.

Real-life example. Skip missing marks represented by null but process the others.

How it works. In a for loop, continue still proceeds to the update expression. In a while loop, be careful not to skip a counter update that you need to reach the stopping condition.

Code to try

```
const marks = [70, null, 80];
for (let i = 0; i < marks.length; i++) {
  if (marks[i] === null) continue;
  console.log(marks[i]);
}
```

Expected result. 70 | 80

Why and when to use it. Use it when some items should be ignored.

Another way or a common risk. Wrapping the remaining work in an if block also works. continue can reduce nesting.

36. Comma operator: evaluate in sequence

What it is. The comma operator evaluates expressions left to right and gives the last value.

Real-life example. Move two counters together during a search from both ends.

How it works. In left++, right-- the comma is an operator joining two update expressions. The comma in let left = 0, right = 2 separates declarations. Commas in arguments and arrays are separators too. They are not all uses of the comma operator.

Code to try

```
for (let left = 0, right = 2;
    left < right;
    left++, right--) {
  console.log(left, right);
}
```

Expected result. 0 2

Why and when to use it. Recognize it in a for-loop update with several actions.

Another way or a common risk. Separate updates or a different loop can be clearer. You rarely need this operator elsewhere.

Practice and explanation

Your task. Which numbers print? Explain the different effects of continue and break.

Predict before running

```
for (let i = 1; i <= 5; i++) {
  if (i === 2) continue;
  if (i === 4) break;
  console.log(i);
}
```

Expected output. 1 | 3

Why. The loop skips printing 2. It prints 3. At 4 it exits completely, so 4 and 5 do not print.

Further reading https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Control_flow_and_error_handling

Part 5 Functions

Functions name reusable work and return values to their callers.

37. Function declaration and call

What it is. Defining a function stores instructions. Calling it with parentheses runs those instructions.

Real-life example. A grading rule can be used for many different students.

How it works. mark and bonus are parameters, the names inside the function. 45 and 10 are arguments, the actual inputs in one call. The arguments fill the parameters by position. The function body does not execute merely because you declare it.

Code to try

```
function addBonus(mark, bonus) {
  return mark + bonus;
}
console.log(addBonus(45, 10));
console.log(addBonus(70, 5));
```

Expected result. 55 | 75

Why and when to use it. Use a function for a task you want to name and reuse.

Another way or a common risk. Repeated copies of the same calculation can drift apart when you edit only one copy.

38. return and console.log

What it is. return sends a result to the caller and stops that call. console.log displays a value.

Real-life example. A calculator must give back a number so another calculation can use it.

How it works. The call double(5) evaluates to 10. The variable result stores 10. result + 3 evaluates to 13. Code after an executed return in that function call does not run. return without a value, or reaching the end, produces undefined.

Code to try

```
function double(n) {
  return n * 2;
}
const result = double(5);
console.log(result + 3);
```

Expected result. 13

Why and when to use it. Return a value when other code needs to use the answer.

Another way or a common risk. A function that only logs and has no return produces undefined as its return value.

39. Local variables and function hoisting

What it is. Each function call has its own local variables. Function declarations can be called earlier in their scope.

Real-life example. Two calculations can use a temporary total without sharing it.

How it works. Hoisting is a description of how bindings become available, not physical movement of source code. The total variable exists inside the function scope. An expression such as const add = function(...) {...} is available only after that assignment executes.

Code to try

```
console.log(add(2, 3));
function add(a, b) {
  const total = a + b;
  return total;
}
```

Expected result. 5

Why and when to use it. Use local names for temporary work. Recognize declaration hoisting when reading code.

Another way or a common risk. Globals can mix unrelated calculations. Calling a const function expression before initialization causes an error.

40. Functions as values

What it is. A function can be stored in a variable, passed to another function or returned.

Real-life example. Give a worker a task instruction to perform.

How it works. sayHello without parentheses refers to the function. sayHello() executes it and produces its return value. This difference explains callbacks and factory functions that return functions. Functions are called first-class values because ordinary value operations can handle them.

Code to try

```
function sayHello() {
  console.log("Hello");
}
const task = sayHello;
task();
```

Expected result. Hello

Why and when to use it. Use this when a program needs to choose which behavior to run.

Another way or a common risk. You can write fixed calls, but the surrounding code cannot accept a different task as easily.

41. Anonymous function expressions

What it is. A function expression can omit the function name after the keyword function. A variable can still refer to it.

Real-life example. Save a small discount calculation under a useful variable name.

How it works. Modern engines often infer the function name from the variable for debugging. The expression still has no written name after function. A standalone declaration normally requires a name.

Code to try

```
const discount = function (price) {
  return price - 20;
};
console.log(discount(100));
```

Expected result. 80

Why and when to use it. Use expressions when assigning or passing a function as a value.

Another way or a common risk. A named function declaration also works. Anonymous syntax is an option, not a requirement.

42. Pass by value: a primitive input

What it is. A parameter receives a value. Reassigning that local parameter does not reassign the outside variable.

Real-life example. A teacher copies your score onto a separate calculation sheet.

How it works. The local mark starts as 40 and becomes 50. score remains 40. To update it, write score = addTen(score). JavaScript always passes arguments by value, including object reference values.

Code to try

```
let score = 40;
function addTen(mark) {
  mark += 10;
  return mark;
}
console.log(addTen(score));
console.log(score);
```

Expected result. 50 | 40

Why and when to use it. Return the changed number and assign it when the caller should keep the result.

Another way or a common risk. Expecting a numeric argument to change automatically leaves the caller with its old value.

43. Object arguments: a shared record

What it is. When the value is an object reference, caller and parameter can reach the same object.

Real-life example. Two people use copies of the address of one student record.

How it works. The reference value is copied. The object itself is shared. This is often loosely called pass by reference, but that wording hides the next distinction: assigning a new object to the parameter does not replace the caller variable.

Code to try

```
const student = {mark: 40};
function addTen(person) {
  person.mark += 10;
}
addTen(student);
console.log(student.mark);
```

Expected result. 50

Why and when to use it. Mutate a property when changing the shared record is intentional.

Another way or a common risk. An unexpected property mutation can surprise the caller. Return a new object if you want a separate result.

44. Reassigning an object parameter

What it is. Changing a parameter to point at a new object does not redirect the outside variable.

Real-life example. Your colleague writes a different address on their own copy of the note. Your note stays the same.

How it works. person.mark = 90 would change the shared original object. person = {mark: 90} changes only person. To replace the caller value, return the new object and assign it to a caller variable declared with let.

Code to try

```
const student = {mark: 40};
function replace(person) {
  person = {mark: 90};
}
replace(student);
console.log(student.mark);
```

Expected result. 40

Why and when to use it. Distinguish changing an object property from changing the local reference.

Another way or a common risk. Mixing the two operations makes function side effects difficult to predict.

45. Default parameters

What it is. A default provides a backup when an argument is missing or undefined.

Real-life example. A purchase has no discount unless the caller supplies one.

How it works. An explicit 0, false or null does not trigger a default parameter. Avoid confusing defaults with || fallbacks, which also replace other falsy values. These two calls are independent.

Code to try

```
function priceAfter(price, discount = 0) {
  return price - discount;
}
console.log(priceAfter(100, 20));
console.log(priceAfter(100));
```

Expected result. 80 | 100

Why and when to use it. Use defaults for optional inputs with a sensible usual value.

Another way or a common risk. You can check for undefined inside the function, but every call otherwise needs extra handling.

46. Recursion: a smaller version of the task

What it is. A recursive function calls itself. A base case stops the repetition.

Real-life example. Count down before a class activity begins.

How it works. Call 3 prints 3 then calls 2. Call 2 prints 2 then calls 1. Call 1 prints 1 then calls 0. Call 0 returns without printing. Each pending call then finishes. The input must move toward the base case.

Code to try

```
function countdown(n) {
  if (n <= 0) return;
  console.log(n);
  countdown(n - 1);
}
countdown(3);
```

Expected result. 3 | 2 | 1

Why and when to use it. Use recursion when a task naturally contains smaller tasks of the same kind, such as nested folders.

Another way or a common risk. A loop handles this simple countdown well. Missing a base case can cause a stack overflow.

47. Recursion with a returned result

What it is. A recursive result can combine the current value with the smaller call result.

Real-life example. Add page counts for numbered lesson packs: pack 3 contributes 3 pages, pack 2 contributes 2 and pack 1 contributes 1.

How it works. Assume n is a nonnegative integer. totalPages(3) waits for 3 + totalPages(2). That waits for 2 + totalPages(1). The base returns 0, then results return as 1, 3 and 6. Very deep recursion can exceed the call stack.

Code to try

```
function totalPages(n) {
  if (n === 0) return 0;
  return n + totalPages(n - 1);
}
console.log(totalPages(3));
```

Expected result. 6

Why and when to use it. Use a recursive definition when the smaller result naturally helps build the larger one.

Another way or a common risk. A loop is often simpler for flat counts. Recursion is optional here.

Practice and explanation

Your task. Which call changes the caller variable? Explain why.

Predict before running

```
function addBonus(mark, bonus = 5) {
  return mark + bonus;
}
let score = 40;
console.log(addBonus(score));
console.log(score);
score = addBonus(score, 10);
console.log(score);
```

Expected output. 45 | 40 | 50

Why. The first call returns 45 but does not reassign score. The final assignment stores the returned 50 in score.

Further reading https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Functions

Part 6 Objects and prototypes

Objects connect data and behavior. Prototypes explain how shared behavior is found.

48. Object methods

What it is. A method is a function stored as an object property.

Real-life example. A student record can calculate its own pass result.

How it works. student.result() calls the method. For this regular method call, this is student. this.mark reads 75. The same operation can serve other student objects without hard-coding their variable names.

Code to try

```
const student = {
  name: "Asha", mark: 75,
  result() {
    return this.mark >= 50 ? "Pass" : "Fail";
  }
};
console.log(student.result());
```

Expected result. Pass

Why and when to use it. Use methods for actions that naturally belong to a record.

Another way or a common risk. A standalone function taking student as an argument also works. Methods group behavior with data.

49. this: the object used for the call

What it is. For a normal object.method() call, this refers to that object.

Real-life example. The same introduction script can speak for different students.

How it works. The call form matters. const f = a.introduce; f() does not automatically keep a as this. A standalone strict function call has undefined this. Arrow functions inherit this from their surrounding scope. Section 8 explains bind and arrow behavior.

Code to try

```
function introduce() {
  return this.name;
}
const a = {name: "Asha", introduce};
const b = {name: "Rafi", introduce};
console.log(a.introduce());
console.log(b.introduce());
```

Expected result. Asha | Rafi

Why and when to use it. Use this to access the receiver of a regular method call.

Another way or a common risk. Hard-coding a.name prevents the same method from naturally working with b.

50. Constructor functions and new

What it is. A constructor initializes a new object when you call it with new.

Real-life example. Use one blank record template for hundreds of students.

How it works. new creates an object, links it to Student.prototype and calls Student with that object as this. Normally it returns the new object. An explicitly returned object can replace that result, an advanced exception. Capitalizing constructor names is a convention. Avoid calling this constructor without new.

Code to try

```
function Student(name, mark) {
  this.name = name;
  this.mark = mark;
}
```

```
const a = new Student("Asha", 75);
console.log(a.name, a.mark);
```

Expected result. Asha 75

Why and when to use it. Use a constructor to make objects with the same structure.

Another way or a common risk. Object literals or factory functions also create records. Copying large literals repeatedly invites inconsistent fields.

51. Prototype: a place to share methods

What it is. An object has an internal prototype link. JavaScript follows it when a property is missing on the object.

Real-life example. Each student has individual marks but can use one shared result rule.

How it works. Object.create(rules) makes a new object whose prototype is rules. student owns mark. result is inherited from rules. During student.result(), this is still student. Inheritance does not mean copying all the prototype properties into the child.

Code to try

```
const rules = {
  result() {
    return this.mark >= 50 ? "Pass" : "Fail";
  }
};
const student = Object.create(rules);
student.mark = 75;
console.log(student.result());
```

Expected result. Pass

Why and when to use it. Use prototypes to share behavior among objects.

Another way or a common risk. Separate per-object functions work too, but identical function copies need not be created.

52. Property lookup and shadowing

What it is. JavaScript checks the object first, then its prototype chain. The first matching property wins.

Real-life example. A personal setting overrides a shared default setting.

How it works. settings first inherits light. Assigning settings.theme creates an own property in this simple writable-property case. It shadows the inherited theme. The original defaults object remains light. The chain ends at null. A missing property after the full search yields undefined.

Code to try

```
const defaults = {theme: "light"};
const settings = Object.create(defaults);
console.log(settings.theme);
settings.theme = "dark";
console.log(settings.theme);
console.log(defaults.theme);
```

Expected result. light | dark | light

Why and when to use it. Use inherited defaults and local overrides when that model matches your task.

Another way or a common risk. Explicit fallback checks can do the same work. Unexpected inheritance can make a value seem to appear from nowhere.

53. Constructor and prototype together

What it is. Constructors store each instance's data. The prototype stores shared methods.

Real-life example. Every student has a different name and mark but uses the same pass rule.

How it works. Student.prototype is a property on the constructor function. Object.getPrototypeOf(a) reads a's internal prototype link. For this instance the result is Student.prototype. An ordinary instance does not need its own property named prototype. Keep mutable per-student data on the instance.

Code to try

```
function Student(name, mark) {
  this.name = name; this.mark = mark;
}
Student.prototype.passed = function () {
  return this.mark >= 50;
};
const a = new Student("Asha", 75);
const b = new Student("Rafi", 40);
console.log(a.passed(), b.passed());
console.log(a.passed === b.passed);
```

Expected result. true false | true

Why and when to use it. Use this pattern when many objects share behavior.

Another way or a common risk. Placing passed inside the constructor creates a fresh function for each object.

54. Prototypal inheritance

What it is. An object can use behavior from another object through a prototype link.

Real-life example. A teaching assistant has a course and can also use a general person introduction.

How it works. assistant owns name and course. The introduce method comes from person. JavaScript looks for introduce on assistant, then person. Because the call receiver is assistant, this.name reads Rafi. Section 7 shows class syntax for a related inheritance structure.

Code to try

```
const person = {
  introduce() { return "I am " + this.name; }
};
const assistant = Object.create(person);
assistant.name = "Rafi";
assistant.course = "Web Tech";
console.log(assistant.introduce());
```

Expected result. I am Rafi

Why and when to use it. Use inheritance when an object should delegate missing behavior to another object.

Another way or a common risk. You can explicitly compose objects or call helper functions. Inheritance is one design choice.

55. globalThis

What it is. globalThis provides a common way to access the global object across JavaScript environments.

Real-life example. A shared environment offers built-in tools such as Math.

How it works. A browser page commonly exposes window. Workers and Node.js have different environments. Ordinary top-level let and const declarations do not automatically become properties of globalThis. Keep application data local unless broad shared state is intentional.

Code to try

```
console.log(globalThis.Math === Math);
console.log(globalThis.Math.max(10, 20));
```

Expected result. true | 20

Why and when to use it. Use it when reusable code needs explicit access to the global environment.

Another way or a common risk. Environment-specific names such as window do not exist in every runtime.

56. Property settings: writable and configurable

What it is. A data property has a value plus flags controlling updates, listing and redefinition.

Real-life example. A record ID can be protected from accidental reassignment.

How it works. writable controls assignment to a data property value. enumerable affects common listings. configurable controls deletion and most redefinition. Omitted flags for a new defineProperty property default to false. A forbidden write can fail silently outside strict mode and throw in strict mode. Getters and setters are accessor properties with different descriptor fields.

Code to try

```
const student = {};
Object.defineProperty(student, "id", {
  value: "S101",
  writable: false,
  enumerable: true,
  configurable: false
});
console.log(student.id);
```

Expected result. S101

Why and when to use it. Use property descriptors when an API needs precise property behavior.

Another way or a common risk. Ordinary properties are usually writable and configurable. That is fine for many records.

57. Enumerable properties

What it is. Enumerable properties appear in common property listings. Non-enumerable does not mean secret.

Real-life example. A report may omit an internal field while code can still read it directly.

How it works. Object.keys returns own enumerable string keys. Properties created through ordinary assignment or a literal normally have enumerable true. Making a property non-enumerable is not access control and does not protect sensitive information.

Code to try

```
const student = {name: "Asha"};
Object.defineProperty(student, "internalId", {
  value: 101, enumerable: false
});
console.log(Object.keys(student));
console.log(student.internalId);
```

Expected result. ["name"] | 101

Why and when to use it. Use enumerability to control listing behavior.

Another way or a common risk. All ordinary literal properties appear in Object.keys. A report may include fields you meant to omit.

58. Own and inherited properties

What it is. Own means stored directly on the object. Inherited means found through its prototype chain.

Real-life example. A student owns a name but inherits a shared school default.

How it works. Ownership and enumerability are independent dimensions. An own property can be enumerable or non-enumerable. Object.hasOwn also works for objects that lack a normal prototype or define their own hasOwnProperty name.

Code to try

```
const shared = {school: "AIUB"};
const student = Object.create(shared);
student.name = "Asha";
console.log(Object.hasOwn(student, "name"));
console.log(Object.hasOwn(student, "school"));
console.log("school" in student);
```

Expected result. true | false | true

Why and when to use it. Use Object.hasOwn when you need to know what the record itself contains.

Another way or a common risk. The in operator also sees inherited properties and may answer a different question than you intended.

59. for…in: list property names

What it is. for…in visits enumerable string keys, including inherited ones.

Real-life example. Print the fields of a student record while excluding shared defaults.

How it works. for…in provides keys, not the values directly. Brackets evaluate the key variable to read each value. for…of instead visits iterable values, such as array items. Symbols do not appear in for…in.

Code to try

```
const shared = {school: "AIUB"};
const student = Object.create(shared);
student.name = "Asha";
for (const key in student) {
  if (Object.hasOwn(student, key)) {
    console.log(key, student[key]);
  }
}
```

Expected result. name Asha

Why and when to use it. Use an ownership check when only the record's own fields should count.

Another way or a common risk. Without the check, this loop also prints school AIUB. Object.keys is another option for own fields.

60. Factory functions

What it is. A factory is an ordinary function that returns a new object.

Real-life example. A registration desk creates a new record for each student.

How it works. This factory creates a new method function for each returned object. Factories can also share functions deliberately. Factory versus constructor is an API choice, not a statement that one is always better.

Code to try

```
function createStudent(name, mark) {
  return {
    name, mark,
    passed() { return this.mark >= 50; }
  };
}
const a = createStudent("Asha", 75);
console.log(a.passed());
```

Expected result. true

Why and when to use it. Use a factory when a plain function that returns an object is easy to understand.

Another way or a common risk. Constructors or literals work too. A factory avoids requiring callers to remember new.

61. Object destructuring

What it is. Destructuring reads named properties into local variables.

Real-life example. Pull only the name and mark from a larger registration record.

How it works. name: label renames the local variable, not the object property. A default applies when the property value is undefined, not null. Destructuring does not deep-clone objects: a nested object value still refers to the original nested object.

Code to try

```
const student = {name: "Asha", mark: 75};
const {name, mark} = student;
console.log(name, mark);
const {name: label, section = "K"} = student;
console.log(label, section);
```

Expected result. Asha 75 | Asha K

Why and when to use it. Use it when you need several properties locally or in function parameters.

Another way or a common risk. const name = student.name works too. Destructuring removes repeated object access.

62. Optional chaining

What it is. ?. stops a property access or call when the value it checks is null or undefined.

Real-life example. Some student records have no address yet.

How it works. ?? provides a fallback for null or undefined. Optional chaining does not suppress arbitrary errors, create undeclared variables or make a non-function callable. Optional calls use method?.(). Use it where absence is expected, rather than hiding a field your program requires.

Code to try

```
const student = {name: "Asha"};
console.log(student.address?.city);
const city = student.address?.city ?? "Unknown";
console.log(city);
```

Expected result. undefined | Unknown

Why and when to use it. Use it for genuinely optional nested data.

Another way or a common risk. Reading student.address.city here throws. Explicit null checks can also handle the missing address.

63. Object literal shortcuts

What it is. Property shorthand reuses variable names. Method shorthand defines actions. Computed keys use a value as a name.

Real-life example. Build a record whose selected mark field comes from a form setting.

How it works. Square brackets around field evaluate it and use "mark" as the property name. Without brackets, field would be a literal property name. The shorthand does not change the data model.

Code to try

```
const name = "Asha";
const field = "mark";
const student = {
  name,
  [field]: 75,
  greet() { return "Hi " + this.name; }
};
console.log(student.mark);
console.log(student.greet());
```

Expected result. 75 | Hi Asha

Why and when to use it. Use these forms to express ordinary objects with less repetition.

Another way or a common risk. Long forms such as name: name and greet: function() also work.

64. Shallow copying an object

What it is. Object spread copies an object's own enumerable properties into a new object. Nested objects still refer to the same nested value.

Real-life example. Edit a student copy without changing the original top-level mark.

How it works. The new object gets copies of the top-level property values. If one value is itself an object, its reference is copied. This operation is a shallow copy, not a recursive clone.

Code to try

```
const original = {name: "Asha", mark: 40};
const updated = {...original, mark: 50};
console.log(original.mark);
console.log(updated.mark);
```

Expected result. 40 | 50

Why and when to use it. Use for simple record updates when you want a new object; inspect nested data separately.

Another way or a common risk. Assigning updated = original would share one object; changing its mark would affect both variables.

Try it yourself. Would updated.name still be Asha?

Check your answer. Yes. The spread copies name, then mark: 50 overrides the copied mark.

Practice and explanation

Your task. Which output comes from inheritance? Which query checks only own properties?

Predict before running

```
const rules = {passed: true};
const student = Object.create(rules);
student.name = "Asha";
console.log(student.passed);
console.log(Object.hasOwn(student, "passed"));
console.log("passed" in student);
console.log(Object.keys(student));
```

Expected output. true | false | true | ["name"]

Why. passed comes from rules. hasOwn excludes inherited properties. in includes them. Object.keys lists the own enumerable name field.

Further reading https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Working_with_objects

Part 7 Classes

Classes give a structured way to create related objects.

65. Classes and instances

What it is. A class defines how to initialize objects and which methods they share. Each created object is an instance.

Real-life example. One Student class creates many individual records.

How it works. Instance methods such as passed are on Student.prototype. Classes still use prototype inheritance. Class bodies run in strict mode, require new for construction and cannot be used before their declaration initializes. A constructor method runs during new.

Code to try

```
class Student {
  constructor(name, mark) {
    this.name = name; this.mark = mark;
  }
  passed() { return this.mark >= 50; }
}
const a = new Student("Asha", 75);
console.log(a.passed());
```

Expected result. true

Why and when to use it. Use a class when a group of related objects has common behavior.

Another way or a common risk. Factories and constructor/prototype code can also model the task. Classes provide a familiar organized syntax.

66. Getters and setters

What it is. A getter computes a value when read. A setter handles assignment and can validate it.

Real-life example. Allow a mark only when it falls between 0 and 100.

How it works. r.mark invokes the getter without parentheses. r.mark = 75 invokes the setter. _mark is a naming convention, not real privacy. Setting this.mark inside this setter would recursively call itself. Production validation should also decide how to handle invalid types and values.

Code to try

```
class Result {
  constructor() { this._mark = 0; }
  get mark() { return this._mark; }
  set mark(value) {
    if (value >= 0 && value <= 100) {
      this._mark = value;
    }
  }
}
const r = new Result();
r.mark = 75;
console.log(r.mark);
```

Expected result. 75

Why and when to use it. Use accessors when property-style reads and writes should run controlled logic.

Another way or a common risk. Explicit getMark and setMark methods are another option. Unchecked assignments can violate your rule.

67. Class expressions

What it is. A class can be a value assigned to a variable.

Real-life example. Choose a record class through an ordinary variable binding.

How it works. Class expressions may be named or anonymous. The variable must be initialized before use. Treat this as another way to express a class rather than an additional object model.

Code to try

```
const Student = class {
  constructor(name) { this.name = name; }
  greet() { return "Hi " + this.name; }
};
const a = new Student("Asha");
console.log(a.greet());
```

Expected result. Hi Asha

Why and when to use it. Use class expressions when assigning or returning a class as a value.

Another way or a common risk. A class declaration is usually simpler when no expression context is needed.

68. Computed class members

What it is. A name inside square brackets comes from evaluating an expression.

Real-life example. A plugin chooses the name of an action when it defines a class.

How it works. The action variable contains "greet" when the class is defined. JavaScript therefore defines a method named greet. Brackets here choose the member name. They do not turn the result into an array.

Code to try

```
const action = "greet";
class Student {
  [action]() { return "Hello"; }
}
const a = new Student();
console.log(a.greet());
```

Expected result. Hello

Why and when to use it. Use computed names when an API determines member names dynamically.

Another way or a common risk. A fixed greet() method is clearer when its name never changes.

69. Class inheritance: extends and super

What it is. extends builds on a parent class. super calls parent initialization or methods.

Real-life example. A teaching assistant is a person with an extra course field.

How it works. A derived constructor must call super before using this. super(name) initializes the parent part on the same instance. The inherited greet method sees the assistant as this. A child may override a parent method and call super.greet() to reuse the parent behavior.

Code to try

```
class Person {
  constructor(name) { this.name = name; }
  greet() { return "Hi " + this.name; }
}
class Assistant extends Person {
  constructor(name, course) {
    super(name);
    this.course = course;
  }
}
const a = new Assistant("Rafi", "Web");
console.log(a.greet());
```

Expected result. Hi Rafi

Why and when to use it. Use inheritance when the child fits the parent model and should reuse its behavior.

Another way or a common risk. Composition and helper functions can avoid a rigid family tree. Choose the relationship deliberately.

70. new.target

What it is. new.target identifies the constructor targeted by a new call. In a plain function call it is undefined.

Real-life example. A reusable constructor can detect whether the caller used new.

How it works. Inside a base constructor reached via a derived construction, new.target can refer to the derived constructor. It is a metaproperty, not a normal property you created on new. Do not confuse it with this, which refers to the instance.

Code to try

```
function Ticket() {
  console.log(new.target === Ticket);
}
```

```
Ticket();
new Ticket();
```

Expected result. false | true

Why and when to use it. Use it for constructor checks and specialized inheritance behavior.

Another way or a common risk. Most beginner classes do not need it. Classes already reject calls without new.

71. Static methods

What it is. A static method belongs to the class itself. It is called with the class name.

Real-life example. A general validation rule does not need an individual student record.

How it works. Call Student.isValidMark, not new Student().isValidMark. Static methods are properties of the constructor class object. They do not automatically have access to instance fields.

Code to try

```
class Student {
  static isValidMark(mark) {
    return Number.isFinite(mark) &&
      mark >= 0 && mark <= 100;
  }
}
console.log(Student.isValidMark(75));
console.log(Student.isValidMark(120));
```

Expected result. true | false

Why and when to use it. Use static methods for class-related utilities or alternate construction helpers.

Another way or a common risk. A separate helper function is also valid. An instance method would require an unnecessary object.

72. Static properties

What it is. A static property holds data on the class rather than on each instance.

Real-life example. All students use one default school label.

How it works. Static fields can be reassigned unless you deliberately restrict them. A static property is not an instance property. A normal instance lookup follows the instance prototype chain rather than searching the constructor for static fields.

Code to try

```
class Student {
  static school = "AIUB";
  constructor(name) { this.name = name; }
}
const a = new Student("Asha");
console.log(Student.school);
console.log(a.school);
```

Expected result. AIUB | undefined

Why and when to use it. Use static fields for information associated with the class as a whole.

Another way or a common risk. A module-level constant can also hold shared information. Repeating a copy on each instance may be unnecessary.

73. Private fields: #name

What it is. A # field is accessible only from code allowed by its declaring class.

Real-life example. A wallet protects its balance from arbitrary outside assignments.

How it works. Writing w.#balance outside the class is a syntax error. Private fields must be declared. They enforce an encapsulation boundary in code, but they are not a replacement for securing data sent to a browser. This deposit example assumes a numeric argument.

Code to try

```
class Wallet {
  #balance = 0;
  deposit(amount) {
    if (amount > 0) this.#balance += amount;
  }
  getBalance() { return this.#balance; }
}
const w = new Wallet();
w.deposit(100);
console.log(w.getBalance());
```

Expected result. 100

Why and when to use it. Use private fields to keep internal state behind controlled methods.

Another way or a common risk. A public field can be changed directly. An underscore prefix is only a convention.

74. Private methods

What it is. A # method keeps an internal helper inside the class.

Real-life example. A wallet uses its own validation helper before accepting a deposit.

How it works. An outside caller cannot invoke w.#valid. The public canDeposit method can call the private helper. Subclasses cannot directly access private names declared by a parent class.

Code to try

```
class Wallet {
  #valid(n) {
    return Number.isFinite(n) && n > 0;
  }
  canDeposit(n) { return this.#valid(n); }
}
const w = new Wallet();
console.log(w.canDeposit(100));
console.log(w.canDeposit(-5));
```

Expected result. true | false

Why and when to use it. Use private methods when outside callers should rely on a smaller public interface.

Another way or a common risk. A public helper works but becomes something other code may depend on.

Practice and explanation

Your task. Separate instance behavior from static data. Predict all outputs.

Predict before running

```
class Student {
  static school = "AIUB";
  constructor(name) { this.name = name; }
  greet() { return "Hi " + this.name; }
}
const a = new Student("Asha");
console.log(a.greet());
console.log(Student.school);
console.log(a.school);
```

Expected output. Hi Asha | AIUB | undefined

Why. a.greet uses a as this. school belongs to Student itself, so a.school does not find it.

Further reading https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Classes

Part 8 Functions in more depth

Functions can be passed, bound, returned and used to remember surrounding state.

75. Function type and properties

What it is. Functions are callable objects. They can have names, properties and methods.

Real-life example. A saved calculation has a name and a declared number of inputs.

How it works. length counts parameters before the first default and excludes a rest parameter. It does not count calls or actual arguments received. The Function constructor can build functions from strings, but ordinary declarations and expressions are clearer and safer for these examples.

Code to try

```
function add(a, b) { return a + b; }
console.log(typeof add);
console.log(add.name);
console.log(add.length);
console.log(add(2, 3));
```

Expected result. function | add | 2 | 5

Why and when to use it. Recognize function values and their metadata when inspecting code.

Another way or a common risk. You can write most functions without inspecting name or length. These properties help tools and debugging.

76. Function versus function call

What it is. A function name refers to instructions. Parentheses execute them and produce the returned result.

Real-life example. Hand someone a recipe, or cook the meal now. Those are different actions.

How it works. task refers to sayHello without running it. message stores the result of running sayHello. task() later executes the same function. This is the foundation of callbacks, bind and functions returned by closures.

Code to try

```
function sayHello() {
  return "Hello";
}
```

```
const task = sayHello;
const message = sayHello();
console.log(typeof task);
console.log(message);
console.log(task());
```

Expected result. function | Hello | Hello

Why and when to use it. Pass the function itself when another part of the program should choose when to call it.

Another way or a common risk. Passing the result by mistake can execute too early or give the receiver text instead of a function.

77. call: choose this and run now

What it is. call invokes a regular function with a chosen this value. Later arguments fill its parameters.

Real-life example. Use the same introduction script for two students.

How it works. The first call argument a becomes this. "Web" becomes course. introduce executes immediately. This explanation assumes an ordinary function. Arrow functions take this from their surrounding scope, so call does not replace an arrow's this.

Code to try

```
function introduce(course) {
  return this.name + " studies " + course;
}
const a = {name: "Asha"};
console.log(introduce.call(a, "Web"));
```

Expected result. Asha studies Web

Why and when to use it. Use call to invoke existing behavior with a particular receiver.

Another way or a common risk. You could attach a method or rewrite the function to accept the object explicitly.

78. apply: provide arguments in an array

What it is. apply chooses this like call, but receives the ordinary arguments in an array or array-like object.

Real-life example. A form has already collected course and section as a list.

How it works. a becomes this. The first array value fills course and the second fills section. apply executes immediately and returns the function result. Spread syntax at a call site expands iterable values into arguments.

Code to try

```
function describe(course, section) {
  return this.name + ": " + course + section;
}
const a = {name: "Asha"};
const details = ["Web ", "K"];
console.log(describe.apply(a, details));
```

Expected result. Asha: Web K

Why and when to use it. Use apply when you already hold the arguments as a list.

Another way or a common risk. call(a, ...details) or another explicit call can also work. apply is an alternative call form.

79. bind: prepare a function for later

What it is. bind returns a new function with a chosen this. It does not call the original immediately.

Real-life example. Prepare Asha's introduction now and let another part of the program run it later.

How it works. The new function wraps greet and remembers the chosen this. Ordinary calls use that bound receiver. Constructor use has special rules, so avoid treating bound this as universal for new calls. bind can also preset leading arguments.

Code to try

```
function greet() {
  return "Hi " + this.name;
}
const a = {name: "Asha"};
const greetAsha = greet.bind(a);
console.log(greetAsha());
```

Expected result. Hi Asha

Why and when to use it. Use bind when a detached method must retain its receiver during an ordinary later call.

Another way or a common risk. A detached method loses its original receiver. A wrapper such as () => a.greet() can be an alternative.

80. call, apply and bind compared

What it is. call and apply execute now. bind gives you a function to execute later.

Real-life example. Choose whether to give an introduction now or save the introduction task.

How it works. This function never reads this, so null has no effect on its arithmetic. bind presets a = 2. The later 3 fills b. This is partial application: prepare part of a call in advance.

Code to try

```
function add(a, b) { return a + b; }
console.log(add.call(null, 2, 3));
console.log(add.apply(null, [2, 3]));
const addTwo = add.bind(null, 2);
console.log(addTwo(3));
```

Expected result. 5 | 5 | 5

Why and when to use it. Read the return value carefully: a result for call/apply, a function for bind.

Another way or a common risk. A plain function call is enough when you do not need to control this or preset arguments.

81. Closure: a function remembers variables

What it is. An inner function can keep access to variables from the place where it was created.

Real-life example. A ticket counter remembers its count between button presses.

How it works. makeCounter runs once and returns a function. next stores that function. The inner function still reaches count after makeCounter returns. It keeps access to the variable binding, not a frozen copy of the original zero.

Code to try

```
function makeCounter() {
  let count = 0;
  return function () {
    count++;
    return count;
  };
}
const next = makeCounter();
console.log(next());
console.log(next());
```

Expected result. 1 | 2

Why and when to use it. Use a closure when behavior needs access to state associated with its creation.

Another way or a common risk. An object field can also store state. A single global counter would mix unrelated counters.

82. Closure walkthrough: two counters

What it is. Each call to the outer function creates a separate local count. Each returned function keeps its own count.

Real-life example. Section K and Section E have independent attendance counters.

How it works. First makeCounter creates count K. Second makeCounter creates count E. k calls update only count K. e updates only count E. The functions are separate closures, even though they came from the same source code.

Code to try

```
function makeCounter() {
  let count = 0;
  return function () { return ++count; };
}
const k = makeCounter();
const e = makeCounter();
console.log(k());
console.log(k());
console.log(e());
console.log(k());
```

Expected result. 1 | 2 | 1 | 3

Why and when to use it. Trace creation separately from later calls. Each creation supplies its own remembered state.

Another way or a common risk. Recreating the counter on every click would reset it to zero each time.

83. A closure can remember a rule

What it is. A returned function can keep an input from the outer call for later use.

Real-life example. Different courses choose different passing marks.

How it works. normal remembers the passMark binding from makeChecker(50). strict remembers a different binding from makeChecker(80). The returned function receives mark later. Keep these two different moments of input clear.

Code to try

```
function makeChecker(passMark) {
  return function (mark) {
    return mark >= passMark;
  };
}
const normal = makeChecker(50);
const strict = makeChecker(80);
console.log(normal(70));
console.log(strict(70));
```

Expected result. true | false

Why and when to use it. Use it to create a customized function once and call it many times.

Another way or a common risk. You could pass passMark on every call. A closure keeps that configuration together with the behavior.

84. IIFE: define and run immediately

What it is. An Immediately Invoked Function Expression runs as soon as its expression is evaluated.

Real-life example. Run a one-time setup calculation with temporary local variables.

How it works. The first parentheses treat the function as an expression. The final () calls it. price and quantity remain local to the function. Its return value becomes total. The spelling is often pronounced iffy.

Code to try

```
const total = (function () {
  const price = 100;
  const quantity = 3;
  return price * quantity;
})();
console.log(total);
```

Expected result. 300

Why and when to use it. Recognize IIFEs in setup code and older scripts that isolate variables.

Another way or a common risk. An ordinary function call, block or module often meets the same need in modern code.

85. Returning several pieces of information

What it is. A function returns one value. An object or array can package several related results.

Real-life example. A report includes both the final mark and the pass decision.

How it works. An array is also possible: return [mark, mark >= 50], then destructure by position. Objects name the fields and often make larger results easier to understand. return a, b uses the comma operator and returns only b, not two values.

Code to try

```
function report(mark) {
  return {mark, passed: mark >= 50};
}
const {mark, passed} = report(75);
console.log(mark, passed);
```

Expected result. 75 true

Why and when to use it. Return an object when the result fields have meaningful names.

Another way or a common risk. Several separate calls may repeat work. Global output variables create unwanted shared state.

86. Arrow functions: a shorter expression

What it is. An arrow function can express a small calculation with fewer words.

Real-life example. Double each score using a short reusable function.

How it works. One simple parameter may omit parentheses. Zero or multiple parameters require parentheses. An expression body implicitly returns its expression. An arrow is a function value, so a const variable must be initialized before you call it.

Code to try

```
const double = number => number * 2;
console.log(double(5));
const add = (a, b) => a + b;
console.log(add(2, 3));
```

Expected result. 10 | 5

Why and when to use it. Use arrows for small functions and callbacks when their this behavior fits.

Another way or a common risk. A regular function expression works too. Shorter syntax is optional.

87. Arrow bodies and object results

What it is. A block body needs return. Parentheses around an object literal distinguish it from a block.

Real-life example. Create a small student record from a name.

How it works. n => { n * 2; } computes an expression but returns nothing. name => ({name}) returns an object. name => {name} is parsed as a block rather than the object-return form you intended.

Code to try

```
const double = n => {
  return n * 2;
};
const makeStudent = name => ({name});
console.log(double(5));
console.log(makeStudent("Asha").name);
```

Expected result. 10 | Asha

Why and when to use it. Use a block for several statements and an expression for one returned calculation.

Another way or a common risk. Forgetting return inside a block makes the function return undefined.

88. When an arrow function is unsuitable

What it is. Arrows inherit this from their surrounding scope. They cannot act as constructors with new.

Real-life example. A student method needs this to mean the student who received the call.

How it works. Arrows also have no own arguments object. Use rest parameters for their inputs. call, apply and bind do not change an arrow's lexical this. Inside a regular method, an arrow callback can be useful precisely because it keeps that method's this.

Code to try

```
const student = {
  name: "Asha",
  greet() { return "Hi " + this.name; }
};
console.log(student.greet());
```

Expected result. Hi Asha

Why and when to use it. Use a regular method when this should come from object.method().

Another way or a common risk. Replacing greet with an arrow does not make this refer to student. The surrounding this would be used.

89. Rest parameters: collect inputs

What it is. A final parameter starting with ... gathers remaining arguments into an array.

Real-life example. A bill can contain two items or twenty items.

How it works. A rest parameter must be last and there can be only one. It is a real array. In older regular functions, arguments is an array-like object representing passed arguments. Rest provides a named array and works with arrows too.

Code to try

```
function total(...prices) {
  let sum = 0;
  for (const price of prices) sum += price;
  return sum;
}
console.log(total(10, 20));
console.log(total(10, 20, 30));
```

Expected result. 30 | 60

Why and when to use it. Use rest when the number of inputs varies.

Another way or a common risk. Accepting an explicit array is another clear API. A fixed parameter list limits the convenient input count.

90. Rest and spread: the two directions

What it is. Rest collects arguments in a definition. Spread expands an iterable at a call site.

Real-life example. Pack individual purchases into one bag, or unpack the bag for a function call.

How it works. In function total(...prices), the dots collect. In total(...basket), the dots expand. The array values become three arguments, which the rest parameter collects again. Spread also appears in arrays and objects with context-specific behavior.

Code to try

```
function total(...prices) {
  let sum = 0;
  for (const p of prices) sum += p;
  return sum;
}
const basket = [10, 20, 30];
console.log(total(...basket));
```

Expected result. 60

Why and when to use it. Identify whether ... appears in the parameter list or the call arguments.

Another way or a common risk. You can accept and process basket as one array instead. Choose a clear interface.

91. Callback: hand over a task

What it is. A callback is a function supplied to other code that calls it.

Real-life example. Give an assistant the instruction to say hello.

How it works. doTask receives the function value, so task refers to sayHello. task() invokes it. doTask(sayHello()) would first execute sayHello, then pass its undefined return value and fail when trying to call that value. Callbacks do not automatically imply asynchronous execution.

Code to try

```
function sayHello() {
  console.log("Hello");
}
function doTask(task) {
  task();
}
doTask(sayHello);
```

Expected result. Hello

Why and when to use it. Use callbacks when the caller should choose the behavior while another function controls execution.

Another way or a common risk. A fixed call inside doTask would always run the same task. Passing behavior makes it reusable.

92. Callback walkthrough: a changing task

What it is. The same receiving function can run different supplied functions.

Real-life example. An assistant can greet someone or say goodbye depending on the instruction you provide.

How it works. First call: task = hello, then task() prints Hello. Second call: task = bye, then task() prints Bye. No timer or delay exists here. Both callbacks run immediately during the outer calls.

Code to try

```
function hello() { console.log("Hello"); }
function bye() { console.log("Bye"); }
function doTask(task) { task(); }
doTask(hello);
doTask(bye);
```

Expected result. Hello | Bye

Why and when to use it. Read the supplied function name as the selected action.

Another way or a common risk. Without a callback you might need separate doHello and doBye wrappers.

93. Synchronous callbacks: filter

What it is. filter calls a predicate for each array item and keeps items whose callback result is truthy.

Real-life example. Keep the passing marks from a class list.

How it works. The callback runs with 40 and returns false, so that value is excluded. It runs with 50 and 80 and returns true, so both are included. filter returns a new array and does not mutate the source array in this example. An arrow can write the same callback as mark => mark >= 50.

Code to try

```
const marks = [40, 50, 80];
const passed = marks.filter(function (mark) {
  return mark >= 50;
});
console.log(passed);
```

Expected result. [50, 80]

Why and when to use it. Use array callbacks for common operations on lists.

Another way or a common risk. A loop and an output array can do the same work. filter expresses the selection goal directly.

94. Asynchronous callbacks: a timer

What it is. A timer schedules a callback for later. The current code continues first.

Real-life example. Set a reminder and continue your work while waiting.

How it works. 1000 is a requested delay in milliseconds. The callback becomes eligible after the delay and may run later if other work blocks execution. setTimeout does not pause the surrounding function. A return inside its callback does not synchronously return from the outer caller.

Code to try

```
console.log("Start");
setTimeout(function () {
  console.log("Reminder");
}, 1000);
console.log("End");
```

Expected result. Start | End | Reminder (later)

Why and when to use it. Use callbacks with APIs that report events or delayed completion.

Another way or a common risk. Running the action immediately would ignore the required delay. Other async APIs use promises, covered after this deck.

95. Putting the ideas together

What it is. A factory makes a customized checker. A callback uses that checker to select records.

Real-life example. A course administrator selects students who meet the chosen pass mark.

How it works. makeRule(50) runs first and returns an arrow function. That function keeps access to limit. filter calls it once per student. Asha passes the test, Rafi does not. The resulting array contains the original Asha object reference. No object clone occurs.

Code to try

```
const students = [
  {name: "Asha", mark: 75},
  {name: "Rafi", mark: 40}
];
function makeRule(limit) {
  return student => student.mark >= limit;
}
const passed = students.filter(makeRule(50));
console.log(passed[0].name);
```

Expected result. Asha

Why and when to use it. Use this pattern when one operation stays the same but its rule changes.

Another way or a common risk. A loop with a fixed threshold works too. Reusable rules help when several courses use different thresholds.

Practice and explanation

Your task. Identify the closure, arrow and callback. Predict the three outputs.

Predict before running

```
function makeChecker(limit) {
  return mark => mark >= limit;
}
const check = makeChecker(50);
console.log(check(45));
console.log(check(50));
console.log([40, 50, 80].filter(check));
```

Expected output. false | true | [50, 80]

Why. The arrow remembers limit = 50 through a closure. filter receives check as a callback. Exactly 50 meets the >= boundary.

Further reading https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Closures

Part 9 Array methods

Array methods help search, select, transform and combine lists.

96. Array methods overview

What it is. An array method is a ready-made operation on a list. Some mutate the original array; others return a new result.

Real-life example. An instructor adds a mark, searches for a student and builds a list of passing marks.

How it works. push changes marks. filter creates a new array and leaves marks as it was after push. A callback tells filter which items to keep.

Code to try

```
const marks = [40, 60];
marks.push(80);
const passing = marks.filter(mark => mark >= 50);
console.log(marks);
console.log(passing);
```

Expected result. [40, 60, 80] | [60, 80]

Why and when to use it. Choose a method based on the result you need: add, find, select, transform or combine.

Another way or a common risk. Without these methods, you can write loops, but repetitive operations take more code and are easier to misread.

Try it yourself. Does filter remove 40 from marks?

Check your answer. No. passing is a new array. marks still contains 40.

97. push and pop

What it is. push adds one or more elements at the end and returns the new length. pop removes and returns the final element. Both mutate the array.

Real-life example. Add a late student mark, then remove the final entry if it was entered by mistake.

How it works. push places 80 after the existing two values, so the length becomes 3. pop returns the removed 80. These operations change marks itself.

Code to try

```
const marks = [40, 60];
console.log(marks.push(80));
console.log(marks.pop());
console.log(marks);
```

Expected result. 3 | 80 | [40, 60]

Why and when to use it. Use for simple additions and removals at the end of a list.

Another way or a common risk. Without them, changing an array length and assigning indexes manually is more work.

Try it yourself. What will marks.length be after both calls?

Check your answer. 2.

98. find and includes

What it is. find returns the first array item whose callback test is true, or undefined if none matches. includes checks whether a value exists.

Real-life example. Find the record for student S2 and check whether a list contains mark 50.

How it works. find passes each item to the callback until it gets a truthy result. includes compares array elements directly and returns a boolean. includes does not search a property inside every object.

Code to try

```
const students = [{id: "S1"}, {id: "S2"}];
const found = students.find(s => s.id === "S2");
console.log(found.id);
console.log([40, 50].includes(50));
```

Expected result. S2 | true

Why and when to use it. Use find for an object matching a rule; use includes for direct membership.

Another way or a common risk. A loop can search too, but find expresses the goal and stops at the first match.

Try it yourself. What does find return if you search for S3?

Check your answer. undefined. Check before reading found.id in that case.

99. filter

What it is. filter visits each item and keeps those whose callback returns a truthy value. It returns a new array.

Real-life example. Get all students who passed without deleting failing records.

How it works. The callback receives 40, then 50, then 80. Its result is false, true and true. filter includes the latter two and leaves the input array untouched.

Code to try

```
const marks = [40, 50, 80];
const passed = marks.filter(mark => mark >= 50);
console.log(passed);
console.log(marks);
```

Expected result. [50, 80] | [40, 50, 80]

Why and when to use it. Use when the desired output is a subset of a list.

Another way or a common risk. Without filter, create an empty array and push each item that satisfies an if statement.

Try it yourself. What would mark > 50 keep instead?

Check your answer. Only 80. Exactly 50 would fail that stricter test.

100. map

What it is. map transforms each item and returns a new array of the same length.

Real-life example. Add five bonus marks to each score.

How it works. map calls the callback once per element and collects each returned result. If the callback forgets to return, the corresponding new element will be undefined.

Code to try

```
const marks = [40, 50];
const finalMarks = marks.map(mark => mark + 5);
console.log(finalMarks);
console.log(marks);
```

Expected result. [45, 55] | [40, 50]

Why and when to use it. Use when each input item becomes one output item.

Another way or a common risk. Without map, use a loop to calculate and push each new value.

Try it yourself. What does marks.map(mark => mark >= 50) produce?

Check your answer. [false, true].

101. forEach

What it is. forEach calls a function for each item. It is useful for an action, such as displaying a line, and returns undefined.

Real-life example. Print a message for each mark in a list.

How it works. The callback receives the element and its index for each visit. return inside that callback does not return from an outer function or stop the full forEach loop.

Code to try

```
const marks = [40, 60];
marks.forEach((mark, index) => {
  console.log(index + 1, mark);
});
```

Expected result. 1 40 | 2 60

Why and when to use it. Use for a side effect such as logging, with care. Use for...of if you need break or await in order.

Another way or a common risk. Using forEach to build a new list requires external state; map or filter often express those tasks more clearly.

Try it yourself. Does forEach itself return [40, 60]?

Check your answer. No. It returns undefined.

102. reduce

What it is. reduce combines array items into one accumulated result. The initial value makes the accumulator's starting point clear.

Real-life example. Calculate the total of all student marks.

How it works. The accumulator starts at 0. Each callback returns a new accumulator: 40, then 100, then 180. With no initial value, empty arrays have different behavior.

Code to try

```
const marks = [40, 60, 80];
const total = marks.reduce((sum, mark) => {
  return sum + mark;
}, 0);
console.log(total);
```

Expected result. 180

Why and when to use it. Use for totals and other combinations that naturally produce one value.

Another way or a common risk. Without reduce, a for loop with let total = 0 is perfectly clear. Use whichever makes the rule easier to read.

Try it yourself. What is the average of these three marks?

Check your answer. 180 / 3 = 60.

103. Choosing an array method

What it is. Choose by the shape of the result: one match, some items, changed items, one combined value, or a side effect.

Real-life example. A student report needs one lookup, a passing list and a total mark.

How it works. The same input array can be processed in several ways. None of these four calls mutates marks. Each callback is a function that the method calls for you.

Code to try

```
const marks = [40, 60, 80];
console.log(marks.find(m => m >= 50));
console.log(marks.filter(m => m >= 50));
console.log(marks.map(m => m + 5));
console.log(marks.reduce((a, b) => a + b, 0));
```

Expected result. 60 | [60, 80] | [45, 65, 85] | 180

Why and when to use it. Use this comparison whenever two array methods seem interchangeable.

Another way or a common risk. Picking the wrong method can give the wrong kind of result: find gives one value; filter gives a list.

Try it yourself. Which method should you use to get every passing mark?

Check your answer. filter.

Further reading https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array

Part 10 The DOM and the page

The DOM lets JavaScript find and change the visible page.

104. DOM and the page tree

What it is. The Document Object Model represents a web page as objects JavaScript can read and change.

Real-life example. A result page contains a heading, a mark input and an area for feedback.

How it works. The browser parses the HTML and creates a document tree. querySelector searches that tree with a CSS selector and returns the first matching element or null. The script follows the heading here, so it can find it.

Code to try

```
<!doctype html>
<h1 id="title">Results</h1>
<script>
  const heading = document.querySelector("#title");
  console.log(heading.textContent);
</script>
```

Expected result. Results (in the console)

Why and when to use it. Use DOM APIs when browser JavaScript needs to inspect or update the visible page.

Another way or a common risk. Without the DOM, a JavaScript calculation can run but does not automatically update a page element.

Try it yourself. What would querySelector("#missing") return?

Check your answer. null.

105. Selecting an element and changing text

What it is. querySelector finds a matching element. textContent sets its displayed text content.

Real-life example. Show a pass result on the page.

How it works. The # prefix means the element with that id. Setting textContent replaces the element's text. It treats a string as text rather than parsing it as HTML.

Code to try

```
<p id="result">Waiting</p>
<script>
  const result = document.querySelector("#result");
  result.textContent = "Pass";
</script>
```

Expected result. The page displays Pass.

Why and when to use it. Use textContent for plain messages from your program, especially user-provided text.

Another way or a common risk. Only logging to the console does not give a typical visitor a visible result.

Try it yourself. What would happen if the id were misspelled?

Check your answer. querySelector would return null; setting null.textContent would throw.

106. Creating and adding an element

What it is. createElement makes a new element object. append adds it to the page.

Real-life example. Add a new student name to a list after registration.

How it works. Creating the li alone does not show it. append inserts it as a child of the ul. textContent is appropriate for a name because it does not parse HTML in the name.

Code to try

```
<ul id="students"></ul>
<script>
  const list = document.querySelector("#students");
  const item = document.createElement("li");
  item.textContent = "Asha";
  list.append(item);
</script>
```

Expected result. The page shows a list item Asha.

Why and when to use it. Use when data determines how many page elements you need.

Another way or a common risk. Without creating an element, you can replace a whole HTML string, but building nodes gives clear control over their content.

Try it yourself. Would the item appear if you omitted list.append(item)?

Check your answer. No. It would exist only in JavaScript memory.

Further reading https://developer.mozilla.org/en-US/docs/Web/API/Document/querySelector

Part 11 Events and forms

Events let the page react to clicks and form submissions.

107. Events and addEventListener

What it is. An event reports an interaction such as a click, input change or form submission. addEventListener runs your callback when it occurs.

Real-life example. A teacher presses Check to show whether a mark passed.

How it works. The browser calls the supplied function after a click on that button. Passing a function is the callback idea again. The code inside the callback runs later, when the event happens.

Code to try

```
<button id="check">Check</button>
<p id="result"></p>
<script>
  document.querySelector("#check")
```

```
  .addEventListener("click", () => {
    document.querySelector("#result")
      .textContent = "Button clicked";
  });
</script>
```

Expected result. Clicking Check displays Button clicked.

Why and when to use it. Use it to connect user actions to JavaScript behavior.

Another way or a common risk. Code that runs only when the page loads cannot respond to a later click.

Try it yourself. Would adding parentheses after the callback function mean it is passed for later?

Check your answer. No. Calling it immediately gives its result instead of the function.

108. Reading a form input

What it is. A form input's value is text. Convert it before numeric comparisons or calculations.

Real-life example. The user types a mark and presses a button.

How it works. The callback reads the current input when clicked, converts it, checks the pass rule and changes the paragraph. Empty input and invalid values need separate validation in a real form.

Code to try

```
<input id="mark" type="number" value="50">
<button id="check">Check</button>
<p id="result"></p>
<script>
  document.querySelector("#check")
    .addEventListener("click", () => {
      const raw = document.querySelector("#mark").value;
      const mark = Number(raw);
      document.querySelector("#result").textContent =
        mark >= 50 ? "Pass" : "Fail";
    });
</script>
```

Expected result. Clicking Check with 50 displays Pass.

Why and when to use it. Use this pattern for user-controlled inputs.

Another way or a common risk. Without conversion, addition can join text and numeric validation can behave unexpectedly.

Try it yourself. What should happen if the user enters 40?

Check your answer. The paragraph shows Fail.

109. Form submission and preventDefault

What it is. A form usually submits and navigates. preventDefault stops that default action so JavaScript can handle it on the page.

Real-life example. Check a student mark without refreshing the page.

How it works. The callback receives an event object. preventDefault cancels the default form submission action; it does not stop all code or cancel other events. The button is a submit button inside this form.

Code to try

```
<form id="form">
  <input id="mark" type="number" value="75">
  <button>Check</button>
</form>
<p id="result"></p>
<script>
  document.querySelector("#form")
    .addEventListener("submit", event => {
      event.preventDefault();
      const mark = Number(document.querySelector("#mark").value);
      document.querySelector("#result").textContent =
        mark >= 50 ? "Pass" : "Fail";
    });
</script>
```

Expected result. Submitting the form shows Pass without navigation.

Why and when to use it. Use submit events for form actions, including pressing Enter.

Another way or a common risk. Without preventDefault, a page may reload or navigate before your visible result remains on screen.

Try it yourself. Why listen to the form rather than only the button click?

Check your answer. A submit listener also handles keyboard submission.

Further reading https://developer.mozilla.org/en-US/docs/Web/API/EventTarget/addEventListener

Part 12 Timers promises and async await

Asynchronous operations finish later. Promises and await help handle their results.

110. Synchronous and asynchronous work

What it is. Synchronous instructions finish before the next line. An asynchronous API arranges for later work while current code can continue.

Real-life example. Set a reminder and continue marking other scripts.

How it works. The timer registers a callback. JavaScript finishes the current code before running that callback after at least the requested delay. Exact timing may be later.

Code to try

```
console.log("Start");
setTimeout(() => console.log("Reminder"), 1000);
console.log("End");
```

Expected result. Start | End | Reminder (later)

Why and when to use it. Use asynchronous APIs for timers, network operations and events.

Another way or a common risk. Waiting in a blocking way would freeze useful work and make an interface feel stuck.

Try it yourself. Does setTimeout pause the next console.log?

Check your answer. No. End prints before Reminder.

111. A Promise describes a future result

What it is. A Promise can be pending, fulfilled with a value, or rejected with a reason. then handles success; catch handles failure.

Real-life example. A server request may produce student data later or fail.

How it works. Promise.resolve makes an already fulfilled promise. Each then returns another promise. Returning a value sends it to the next then. Throwing an error or returning a rejected promise moves to catch. Callbacks in then run in a later microtask.

Code to try

```
Promise.resolve(75)
  .then(mark => mark >= 50 ? "Pass" : "Fail")
  .then(result => console.log(result))
  .catch(error => console.error(error));
```

Expected result. Pass

Why and when to use it. Use with APIs that return promises, such as fetch. Do not wrap ordinary synchronous work in a Promise without a reason.

Another way or a common risk. Returning a future result lets code compose operations without blocking the whole program.

Try it yourself. What is passed to the second then?

Check your answer. The string Pass.

112. async and await

What it is. An async function always returns a Promise. await waits for a Promise result inside that function without blocking the whole runtime.

Real-life example. Wait for a mark lookup, then show its result.

How it works. showResult begins and reaches await. Its continuation is scheduled after the resolved promise; the outer script prints Requested first. await resumes with 75. A rejection throws inside the async function.

Code to try

```
async function showResult() {
  const mark = await Promise.resolve(75);
  console.log(mark >= 50 ? "Pass" : "Fail");
}
showResult();
console.log("Requested");
```

Expected result. Requested | Pass

Why and when to use it. Use inside async functions for code that depends on promise results. Top-level await also works in modules.

Another way or a common risk. Chained then calls work, but a sequence of dependent operations can be easier to read with await.

Try it yourself. Does showResult() return the text Pass directly?

Check your answer. No. It returns a Promise; the function logs Pass in this example.

113. Handling errors in async functions

What it is. Use try and catch around awaited operations that can fail. Keep the error visible or show a useful message.

Real-life example. A mark lookup fails when a service is unavailable.

How it works. The awaited rejected promise throws at the await expression. catch receives the Error. A real application might also log error details for diagnosis while showing a clear user message.

Code to try

```
async function loadMark() {
  try {
    const mark = await Promise.reject(new Error("Unavailable"));
    console.log(mark);
  } catch (error) {
    console.log("Could not load mark");
  }
}
loadMark();
```

Expected result. Could not load mark

Why and when to use it. Use for operations where failure is possible, especially network requests.

Another way or a common risk. Without handling rejection, users may see no useful feedback and the runtime reports an unhandled failure.

Try it yourself. Would the first console.log(mark) run here?

Check your answer. No. Execution jumps to catch.

114. Parallel independent requests

What it is. Promise.all waits for every supplied promise and fulfills with results in input order. It rejects if one rejects.

Real-life example. Load a student name and mark at the same time when neither depends on the other.

How it works. Both promises are supplied together. Promise.all fulfills when all do and keeps the supplied order in its result array, even if completion order differs. For a result from every success or failure, see Promise.allSettled.

Code to try

```
async function show() {
  const [name, mark] = await Promise.all([
    Promise.resolve("Asha"),
    Promise.resolve(75)
  ]);
  console.log(name, mark);
}
show();
```

Expected result. Asha 75

Why and when to use it. Use for independent operations that are all required before continuing.

Another way or a common risk. Awaiting independent operations one by one can add unnecessary waiting.

Try it yourself. Would the result order change if the second task finished first?

Check your answer. No. The output array follows input order.

Further reading https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Using_promises

Part 13 Fetching data

fetch connects a page or program to resources and APIs.

115. fetch and a response

What it is. fetch makes an HTTP request and returns a Promise for a Response. A second step reads the response body.

Real-life example. Download a JSON student record from a service.

How it works. This self-contained data URL is used so the example runs without an external server. response.ok checks HTTP success; fetch generally rejects for network errors, while HTTP 404 still gives a Response that you should inspect. response.json() also returns a Promise.

Code to try

```
async function loadStudent() {
  const response = await fetch(
    "data:application/json,%7B%22name%22%3A%22Asha%22%7D"
  );
  if (!response.ok) throw new Error("HTTP error");
  const student = await response.json();
  console.log(student.name);
}
loadStudent();
```

Expected result. Asha

Why and when to use it. Use fetch for resources and APIs; check errors and the response format.

Another way or a common risk. A page cannot obtain remote data simply by declaring a local variable. fetch connects it to a resource.

Try it yourself. Would response.json() immediately be a plain object without await?

Check your answer. No. It returns a Promise.

116. A real API request pattern

What it is. A real application requests an endpoint supplied by its server. The result should be validated before updating the page.

Real-life example. An instructor dashboard requests the current class list.

How it works. The relative URL refers to the same site. This example requires a server that actually provides /api/students and returns JSON. Its output cannot be predetermined without that server. CORS can limit cross-origin requests.

Code to try

```
async function loadStudents() {
  try {
    const response = await fetch("/api/students");
    if (!response.ok) throw new Error(`HTTP ${response.status}`);
    const students = await response.json();
    console.log(students);
  } catch (error) {
    console.error("Could not load students", error);
  }
```

```
}
loadStudents();
```

Expected result. Depends on the server at /api/students.

Why and when to use it. Adapt the URL and expected data shape to a real API you control or are authorized to use.

Another way or a common risk. Without explicit response checks, a 404 page can be mistakenly parsed as expected student JSON.

Try it yourself. What happens if /api/students is absent?

Check your answer. response.ok is false for an HTTP 404, so catch logs an error.

Further reading https://developer.mozilla.org/en-US/docs/Web/API/Window/fetch

Part 14 Errors and modules

Errors report failures. Modules organize code across files.

117. Throwing and catching an error

What it is. throw reports a failure from code. try...catch lets a caller handle it. finally runs after either outcome.

Real-life example. Reject a mark below 0 instead of silently recording an impossible score.

How it works. throw stops the current normal path. The nearest suitable catch receives the Error. An error object has a message and stack for debugging. Catch only where you can respond meaningfully or add context.

Code to try

```
function checkMark(mark) {
  if (mark < 0 || mark > 100) {
    throw new RangeError("Mark must be 0 to 100");
  }
  return mark;
}
try {
  console.log(checkMark(120));
} catch (error) {
  console.log(error.message);
}
```

Expected result. Mark must be 0 to 100

Why and when to use it. Use for exceptional invalid conditions, not routine Pass or Fail decisions.

Another way or a common risk. A misleading result is harder to fix than a clear failure at the point of invalid input.

Try it yourself. What would checkMark(80) return?

Check your answer. 80, and the catch block would not run.

118. Modules with export and import

What it is. A module is a file with its own scope that can export values for other modules to import.

Real-life example. Keep grading rules in one file and page code in another.

How it works. The named export makes getResult available. The import path is relative to app.js and includes the file extension in this browser example. Load app.js with <script type="module" src="app.js"></script> from a local development server.

Code to try

```
// grading.js
export function getResult(mark) {
  return mark >= 50 ? "Pass" : "Fail";
}

// app.js
import {getResult} from "./grading.js";
console.log(getResult(75));
```

Expected result. Pass when app.js runs as a module.

Why and when to use it. Use modules once your code has separable responsibilities and multiple files.

Another way or a common risk. Without modules, a growing project tends to have one large file or shared global names.

Try it yourself. Would import {result} work with only getResult exported?

Check your answer. No. The exported name is getResult unless you use an alias.

119. Modules and local development

What it is. Browser modules normally load over HTTP, so use a local development server rather than relying on file:// URLs.

Real-life example. Open an index page that imports the grading module.

How it works. Save index.html, app.js and grading.js in the same folder. Run a local development server, such as the editor's development server, then open its http://localhost URL. The previous lesson defines grading.js.

Code to try

```
<!-- index.html -->
<!doctype html>
<html><body>
  <script type="module" src="./app.js"></script>
</body></html>

// app.js
import {getResult} from "./grading.js";
console.log(getResult(75));
```

Expected result. Pass in the browser console, if grading.js is present.

Why and when to use it. Use this setup when teaching or developing browser modules.

Another way or a common risk. Opening files directly may cause module loading restrictions and confusing errors.

Try it yourself. Which file needs type="module" in the HTML?

Check your answer. The script entry, app.js. Its imports are then modules too.

Further reading https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Modules

Part 15 Browser storage

Browser storage keeps small local values across reloads.

120. localStorage and strings

What it is. localStorage stores string values for an origin across page reloads. Read with getItem and write with setItem.

Real-life example. Remember the preferred passing threshold between visits.

How it works. The browser stores the key and string value for its origin. getItem returns a string or null. Number(null) is 0, so check missing values before converting if zero has a meaning. Storage can fail or be unavailable in restricted contexts.

Code to try

```
localStorage.setItem("passMark", "50");
const saved = localStorage.getItem("passMark");
console.log(Number(saved));
```

Expected result. 50

Why and when to use it. Use for small, non-sensitive preferences. Do not store passwords or secrets here.

Another way or a common risk. A normal variable disappears when the page reloads. Storage can preserve small settings locally.

Try it yourself. What is typeof saved?

Check your answer. string.

121. Saving an object as JSON

What it is. JSON.stringify turns data into a string. JSON.parse turns that string back into data.

Real-life example. Save a small draft list of student marks in the browser.

How it works. JSON supports data values such as plain objects, arrays, strings, numbers, booleans and null. It does not preserve methods or some special JavaScript values. Parse only data you trust or handle errors. Storage is synchronous and small.

Code to try

```
const students = [{name: "Asha", mark: 75}];
localStorage.setItem("students", JSON.stringify(students));
const saved = localStorage.getItem("students");
const restored = JSON.parse(saved);
console.log(restored[0].name);
```

Expected result. Asha

Why and when to use it. Use for modest local data, not as a secure database or a replacement for server storage.

Another way or a common risk. Storing an object directly produces an unhelpful string representation.

Try it yourself. Will a function method on a student object survive JSON conversion?

Check your answer. No. JSON stores data, not functions.

Further reading https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage

Part 16 Complete class results project

This project combines the earlier concepts into one working student result tracker.

122. Project overview

What it is. Build a page that accepts a name and mark, validates the input, adds a student, shows the class list and remembers it locally.

Real-life example. A small classroom result tracker uses everything from variables through events and storage.

How it works. The plan separates data, logic and display. The next entries give a complete HTML file and script. Test marks at 0, 49, 50 and 100. Check missing names and invalid marks too.

Code to try

```
Input: name and mark
Validate: name, 0 to 100 mark
Store: array of student objects
Display: list and passing count
Persist: localStorage
```

Expected result. A working page updates after each addition and after reload.

Why and when to use it. Use as a capstone after DOM, events and storage.

Another way or a common risk. A project exposes gaps between knowing a syntax form and using it in a full flow.

Try it yourself. Which parts should be pure functions?

Check your answer. A pass check and a count can be pure functions; DOM updates and storage are side effects.

123. Project file one index html

What it is. HTML provides the controls and result locations that JavaScript will use.

Real-life example. A teacher enters a name and mark and reads the updated class list.

How it works. Save this as index.html next to app.js. Labels identify fields. HTML validation helps users, but app.js also checks input because data can be changed by script or other clients. defer lets the script run after parsing the document.

Code to try

```
<!doctype html>
<html lang="en">
<head><meta charset="utf-8"><title>Class Results</title></head>
<body>
  <h1>Class Results</h1>
  <form id="studentForm">
    <label>Name <input id="name" required></label>
    <label>Mark <input id="mark" type="number" min="0" max="100" required></label>
    <button>Add student</button>
  </form>
  <p id="message"></p>
  <ul id="list"></ul>
  <p id="summary"></p>
```

```
  <script src="app.js" defer></script>
</body>
</html>
```

Expected result. A page with a form, list and summary.

Why and when to use it. Use this as the first of two files for the capstone.

Another way or a common risk. Without matching IDs, the script cannot select the elements it expects.

Try it yourself. What must the form id be for app.js to find it?

Check your answer. studentForm.

124. Project file two app js

What it is. The script loads saved data, listens for submissions, validates input, updates the array, saves it and redraws the view.

Real-life example. Each new student appears with a Pass or Fail result.

How it works. Save as app.js in the same folder as index.html. render empties and rebuilds the list using textContent to treat a name as text. The handler validates and stores data before calling render. The final render shows any saved records on page load. This learning project assumes storage contains valid JSON; a production version should handle corrupt stored data and storage errors.

Code to try

```
const form = document.querySelector("#studentForm");
const list = document.querySelector("#list");
const message = document.querySelector("#message");
const summary = document.querySelector("#summary");
let students = JSON.parse(localStorage.getItem("students") || "[]");

function result(mark) { return mark >= 50 ? "Pass" : "Fail"; }
function render() {
  list.replaceChildren();
  for (const student of students) {
    const item = document.createElement("li");
    item.textContent = `${student.name}: ${student.mark} ${result(student.mark)}`;
    list.append(item);
  }
  const passed = students.filter(s => s.mark >= 50).length;
  summary.textContent = `${passed} of ${students.length} passed` ;
}
form.addEventListener("submit", event => {
  event.preventDefault();
  const name = document.querySelector("#name").value.trim();
  const markText = document.querySelector("#mark").value;
  const mark = Number(markText);
  if (!name || markText === "" || !Number.isFinite(mark) || mark < 0 || mark > 100) {
    message.textContent = "Enter a name and a mark from 0 to 100";
    return;
  }
  students.push({name, mark});
  localStorage.setItem("students", JSON.stringify(students));
  message.textContent = "Student added";
  form.reset();
  render();
});
render();
```

Expected result. Adding Asha with 75 shows Asha: 75 Pass and 1 of 1 passed.

Why and when to use it. Use it to review the whole data-to-page flow. Run each file in a browser, not a raw console.

Another way or a common risk. Without validation, blank and out-of-range values could enter the report. Without render after adding, the page would show stale data.

Try it yourself. Change pass mark 50 to 60 in result and in filter. Why both?

Check your answer. The result label and passing count currently use separate checks; move the threshold into a shared function or constant to avoid inconsistency.

125. Project improvement and testing

What it is. Test boundary values, missing input, reload persistence and changing one rule in one place.

Real-life example. A mark of exactly 50 should match the stated policy, and a blank mark should be rejected.

How it works. Replace the duplicated comparisons in result and filter with isPass. Then test 0, 49, 50, 100 and 101, an empty name, an empty mark, and reload after saving. Clear the browser's stored students key when you need a fresh run.

Code to try

```
const PASS_MARK = 50;
function isPass(mark) { return mark >= PASS_MARK; }
console.log(isPass(49));
console.log(isPass(50));
```

Expected result. false | true

Why and when to use it. Use these checks before showing a project to others or building more features.

Another way or a common risk. Two separate hard-coded pass rules can disagree after one is edited.

Try it yourself. What is the result at exactly 50?

Check your answer. true for the pass check.

Further reading https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Scripting